RFID ATTENDANCE MANAGEMENT SYSTEM

Complete Setup & Deployment Guide

Project: RFID Attendance & Access Management System | Platform: Arduino + Python + XAMPP

---

1. Project Overview

This is an RFID-based Employee Attendance and Access Management System built for an institutional or organizational setting (identified by the Our Lady of Fatima University branding). It combines an Arduino microcontroller with a Python GUI desktop application and a local MySQL database to create a complete, offline-capable time-in/time-out tracking system.

1.1 What the System Does

- Reads RFID cards/tags using an MFRC522 RFID reader connected to an Arduino Uno.
- Displays card scan feedback (ACCESS GRANTED or UNREGISTERED) on a 16x2 I2C LCD screen.
- Triggers a buzzer and LED on access granted or denied events.
- Communicates with a Python desktop application over USB Serial (COM6).
- Shows an employee profile popup with Time In / Time Out tracking.
- Logs all scan events (UID, Name, Employee ID, Position, Timestamp) to a local MySQL database via XAMPP.
- Stores employee card-to-name mappings offline in a users.json file.
- Allows admin to register new cards and unregister existing employees from the GUI.

1.2 System Architecture

[RFID Card] --> [Arduino Uno + MFRC522] --> [USB Serial (COM6)] --> [Python GUI App] --> [MySQL via XAMPP]

School	Our Lady of Fatima University (OLFU)

Project Name	RFID Attendance & Access Management System

Serial Port	COM6 at 9600 baud

Database	rfid_db (MySQL via XAMPP/phpMyAdmin)

Python Version	Python 3.12 (confirmed by .pyc cache)

2. Project Files

The project folder RFID Attendance & Access Management System contains the following files:

File	Purpose
`gui_reader.py`	Main Python GUI application (Tkinter) — the primary program to run.
`python_reader.py`	Alternate CLI-only version of the reader (no GUI). For terminal use.
`arduino_code/arduino_code.ino`	Arduino sketch — handles RFID reading, LCD, buzzer, and LED.
`database_setup.sql`	SQL command to manually create the rfid_scans table in phpMyAdmin.
`users.json`	Offline employee registry — maps card UIDs to name, employee ID, and position.
`INSTRUCTION.txt`	Original brief setup notes from the developer.
`OLFU_official_logo.png`	School logo displayed in the GUI header and employee profile popup.
`logo.png.png`	Alternate logo file (also used as fallback by the app).

3. Hardware Requirements

The following physical hardware components are required to build and run the system:

Component	Notes
Arduino Uno	Microcontroller board. Connected to PC via USB (becomes COM6).
MFRC522 RFID Reader Module	Connected to Arduino via SPI pins (SS=Pin 10, RST=Pin 9).
RFID Cards / Key Fobs	ISO 14443A standard cards or tags. Each has a unique 4-byte UID.

16x2 I2C LCD Display	I2C address 0x27. Connected via SDA/SCL pins. Shows scan status.
LED (any color)	Connected to Arduino Pin 2. Lights up on card scan.
Buzzer (active)	Connected to Arduino Pin 3. Beeps on access granted or denied.
USB Cable (Type A to B)	Connects Arduino Uno to the PC. Provides power and serial communication.
Jumper Wires	For wiring components together on a breadboard.
Breadboard (optional)	For prototyping the circuit connections.
PC / Laptop (Windows)	Runs the Python GUI application. Requires a USB port and COM port.

4. Complete Wiring Guide

Wire all components to the Arduino Uno exactly as shown below. Double-check every connection before powering on. Wrong wiring can damage the MFRC522 module (it runs on 3.3V, NOT 5V).

4.1 MFRC522 RFID Reader Module → Arduino Uno (SPI)

IMPORTANT: The MFRC522 operates at 3.3V. Always connect VCC to the 3.3V pin on the Arduino, never to 5V — doing so will permanently damage the module.

MFRC522 Pin	Arduino Uno Pin	Notes
VCC (3.3V)	**3.3V**	Power supply — USE 3.3V ONLY. 5V will destroy the module.
GND	**GND (any)**	Ground. Connect to any GND pin on the Arduino.
RST	**Pin 9**	Reset pin. Defined as RST_PIN 9 in the code.
SDA (SS)	**Pin 10**	SPI Slave Select. Defined as SS_PIN 10 in the code.
MOSI	**Pin 11**	SPI Master Out Slave In. Fixed SPI pin on Arduino Uno.
MISO	**Pin 12**	SPI Master In Slave Out. Fixed SPI pin on Arduino Uno.
SCK	**Pin 13**	SPI Clock. Fixed SPI pin on Arduino Uno.
IRQ	**Not Connected**	Interrupt pin — leave this pin unconnected (floating).

4.2 16x2 I2C LCD Display → Arduino Uno

The I2C LCD uses only 4 wires — VCC, GND, SDA, and SCL. The I2C module (backpack) is usually already soldered onto the LCD. The default I2C address used in the code is 0x27. If your

LCD does not display anything, the address may be 0x3F — change it in arduino_code.ino if needed.

LCD I2C Pin	Arduino Uno Pin	Notes
VCC	**5V**	Power supply for the LCD. Uses 5V unlike the MFRC522.
GND	**GND (any)**	Ground. Connect to any GND pin on the Arduino.
SDA	**Pin A4**	I2C Data line. Fixed I2C SDA pin on Arduino Uno.
SCL	**Pin A5**	I2C Clock line. Fixed I2C SCL pin on Arduino Uno.

4.3 LED → Arduino Uno

An LED is wired to Pin 2. Always use a current-limiting resistor (220Ω recommended) in series with the LED to prevent burning it out.

LED Leg	Arduino Uno Pin	Notes
Anode (+) / Long leg	**Pin 2 → via 220Ω resistor**	The longer leg is positive. Always place a 220Ω resistor between Pin 2 and the anode.
Cathode (−) / Short leg	**GND (any)**	The shorter leg goes directly to any GND pin on the Arduino.

4.4 Active Buzzer → Arduino Uno

An active buzzer (not passive) is wired to Pin 3. Active buzzers produce sound on their own when power is applied — no PWM tone needed. The code simply sets Pin 3 HIGH to beep and LOW to stop.

Buzzer Pin	Arduino Uno Pin	Notes
Positive (+) / Marked side	**Pin 3**	The side marked with '+' or a longer leg connects to Pin 3.
Negative (−)	**GND (any)**	Connect to any GND pin on the Arduino.

4.5 Master Wiring Summary (All Components at a Glance)

Component	Component Pin	Arduino Uno Pin	Color Suggestion
MFRC522	VCC	**3.3V**	Red
MFRC522	GND	**GND**	Black
MFRC522	RST	**Pin 9**	Orange
MFRC522	SDA (SS)	**Pin 10**	Yellow

MFRC522	MOSI	**Pin 11**	Green
MFRC522	MISO	**Pin 12**	Blue
MFRC522	SCK	**Pin 13**	Purple
LCD I2C	VCC	**5V**	Red
LCD I2C	GND	**GND**	Black
LCD I2C	SDA	**Pin A4**	White
LCD I2C	SCL	**Pin A5**	Gray
LED	Anode (+)	**Pin 2 (via 220Ω)**	Any color
LED	Cathode (−)	**GND**	Black
Buzzer	Positive (+)	**Pin 3**	Red
Buzzer	Negative (−)	**GND**	Black

4.6 Wiring Tips & Common Mistakes

- **NEVER connect MFRC522 VCC to 5V. It will burn out the module instantly. Always use the 3.3V pin.**
- Make sure the Arduino Serial Monitor is CLOSED before running the Python app — both cannot use the serial port at the same time.
- The I2C LCD address in the code is 0x27. If your LCD shows nothing or only black squares, try changing it to 0x3F in arduino_code.ino on this line: LiquidCrystal_I2C lcd(0x27, 16, 2);
- Use a 220-ohm resistor between Pin 2 and the LED anode. Without a resistor, the LED may burn out or damage the Arduino pin.
- Verify all GND connections share a common ground. All components — MFRC522, LCD, LED, Buzzer — must share the same GND rail.
- After uploading the Arduino sketch, listen for a single startup beep from the buzzer. This confirms Pin 3 is wired correctly and the sketch uploaded successfully.
- Double-check SPI pins: MOSI=11, MISO=12, SCK=13. These are fixed SPI pins on the Arduino Uno and cannot be changed in software.

5. Software Requirements

4.1 Applications to Install

Install all of the following before proceeding:

Application	Where to Get It	Purpose
Python 3.12+	python.org/downloads	Runs the GUI application (gui_reader.py).

XAMPP	apachefriends.org	Provides the local MySQL database and phpMyAdmin interface.
Arduino IDE	arduino.cc/en/software	Used to upload the arduino_code.ino sketch to the Arduino board.
CH340 / CP2102 Driver	From Arduino board vendor	Required if Windows does not auto-detect the Arduino COM port.

4.2 Arduino IDE Libraries to Install

Open Arduino IDE, go to Sketch > Include Library > Manage Libraries, then search and install each of the following:

- **MFRC522 — by GithubCommunity. Handles RFID card reading via SPI.**
- **LiquidCrystal I2C — by Frank de Brabander. Controls the I2C LCD display.**
- **LiquidCrystal — Built-in with Arduino IDE. May be needed as a base dependency.**

Note: The SPI.h and Wire.h libraries are built into the Arduino IDE and do not need to be installed separately.

4.3 Python Libraries to Install

Open Command Prompt or Terminal and run this single install command:

```
python -m pip install pyserial mysql-connector-python Pillow
```

Python Library	What It Does
`pyserial`	Reads data from the Arduino over USB (COM port serial communication).
`mysql-connector-python`	Connects the Python app to the local MySQL database (XAMPP).
`Pillow`	Loads and displays the OLFU logo PNG image inside the Tkinter GUI.
`tkinter`	GUI framework (built into Python — no install needed).
`json`	Reads/writes the users.json offline employee registry (built-in).
`datetime`	Handles timestamps for Time In / Time Out (built-in).

6. Complete Step-by-Step Setup Procedure

Follow all steps below in order. Do not skip any step.

PART A: Install All Required Software

#	Step	Details / Command
1	**Install Python 3.12+**	Go to python.org/downloads. Download and run the installer. IMPORTANT: Check the box that says 'Add Python to PATH' before clicking Install.
2	**Install XAMPP**	Go to apachefriends.org and download XAMPP. Run the installer. You only need the Apache and MySQL modules — uncheck everything else to save space.
3	**Install Arduino IDE**	Go to arduino.cc/en/software. Download Arduino IDE 2.x. Run the installer and launch the IDE at least once to complete setup.
4	**Install Arduino Drivers (if needed)**	If Windows does not recognize your Arduino when plugged in, download and install the CH340 or CP2102 driver from your board vendor's website.

PART B: Install Arduino Libraries

#	Step	Details / Command
5	**Open Arduino IDE**	Launch the Arduino IDE on your computer.
6	**Open Library Manager**	Go to Sketch > Include Library > Manage Libraries... (or press Ctrl+Shift+I).
7	**Install MFRC522 Library**	Search for: MFRC522. Find the one by GithubCommunity. Click Install.
8	**Install LiquidCrystal I2C**	Search for: LiquidCrystal I2C. Find the one by Frank de Brabander. Click Install.
9	**Verify Libraries**	Go to Sketch > Include Library — you should now see MFRC522 and LiquidCrystal I2C listed.

PART C: Upload Arduino Sketch

#	Step	Details / Command
10	**Connect Arduino to PC**	Plug the Arduino Uno into your computer using the USB cable. Windows will assign it a COM port (e.g., COM6).
11	**Find the COM Port**	Open Device Manager (Win+X > Device Manager). Look under Ports (COM & LPT). Note the COM number assigned to your Arduino.
12	**Open the Arduino Sketch**	In Arduino IDE, go to File > Open. Navigate to: RFID Attendance & Access Management System/arduino_code/arduino_code.ino and open it.

13	**Select the Board**	Go to Tools > Board > Arduino AVR Boards > Arduino Uno.
14	**Select the Port**	Go to Tools > Port and select the COM port your Arduino is on (e.g., COM6).
15	**Upload the Sketch**	Click the Upload button (right arrow icon). Wait for 'Done uploading' in the status bar.
16	**Verify Startup Beep**	After uploading, the buzzer on Pin 3 should produce one short beep. The LCD should display 'Scan your card'. This confirms the hardware is wired and working correctly.
17	**CLOSE the Serial Monitor**	If the Serial Monitor is open in Arduino IDE, CLOSE IT NOW. The Python app needs exclusive access to the serial port — leaving it open will cause a connection error.

PART D: Setup the Database (XAMPP + MySQL)

#	Step	Details / Command
18	**Start XAMPP**	Open the XAMPP Control Panel. Click START next to both Apache and MySQL.
19	**Open phpMyAdmin**	Click the Admin button next to MySQL, or open your browser and go to: http://localhost/phpmyadmin
20	**Create the Database**	In phpMyAdmin, click New in the left sidebar. Type rfid_db as the database name. Choose utf8_general_ci as the collation. Click Create.
21	**Create the Table**	Select the rfid_db database. Click the SQL tab at the top. Paste the following SQL command and click Go: CREATE TABLE IF NOT EXISTS rfid_scans (id INT AUTO_INCREMENT PRIMARY KEY, uid VARCHAR(50) NOT NULL, name VARCHAR(100), emp_id VARCHAR(50), position VARCHAR(100), scan_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP);
22	**Verify Table**	Click on rfid_db in the left panel. You should see rfid_scans listed. Click it to confirm it has the correct columns: id, uid, name, emp_id, position, scan_time.

Note: If you skip Step 21, the Python GUI will auto-create the table when it connects — but running the SQL manually is recommended to avoid column-ordering issues.

PART E: Install Python Libraries

#	Step	Details / Command
23	**Open Command Prompt**	Press Win+R, type cmd, and press Enter. Or search 'Command Prompt' in the Start menu.

24	**Install All Libraries**	Type the following command exactly and press Enter:

```
python -m pip install pyserial mysql-connector-python Pillow
```

#	Step	Details / Command
25	**Wait for Installation**	pip will download and install all three libraries. You should see 'Successfully installed' messages for each.
26	**Verify Installation (Optional)**	Run this command to confirm all libraries are installed:

```
python -m pip list
```

Confirm that pyserial, mysql-connector-python, and Pillow appear in the list.

PART F: Configure COM Port (If Different from COM6)

The code is configured to use COM6 by default. If your Arduino is on a different port:

#	Step	Details / Command
27	**Check Your COM Port**	In Device Manager > Ports (COM & LPT), note your Arduino's port (e.g., COM3, COM4, COM8, etc.).
28	**Edit gui_reader.py**	Open RFID Attendance & Access Management System/gui_reader.py in any text editor (Notepad, VS Code, etc.).
29	**Find the Port Line**	Search for the line: self.ser = serial.Serial('COM6', 9600, timeout=0.1)
30	**Change COM6**	Replace COM6 with your actual port number. Example: self.ser = serial.Serial('COM3', 9600, timeout=0.1). Save the file.

PART G: Run the Application

#	Step	Details / Command
31	**Open Command Prompt**	Press Win+R, type cmd, and press Enter.
32	**Navigate to Project Folder**	Type the following command and press Enter (adjust the path if you saved it elsewhere):

```
cd C:\Users\user\Downloads\RFID Attendance & Access Management System\RFID
Attendance & Access Management System
```

#	Step	Details / Command

33	**Run the Application**	Type the following command and press Enter:

```
python gui_reader.py
```

#	Step	Details / Command
34	**Confirm GUI Launches**	The RFID Management System window should open. The footer bar should show: Arduino Connected (COM6) in green. The header should show: Database Status: Connected (Local).
35	**Test with an RFID Card**	Tap a registered card from users.json on the MFRC522 reader. The LCD should show ACCESS GRANTED, the buzzer should beep once, and an employee profile popup should appear in the GUI.
36	**Test Unregistered Card**	Tap an unknown card. The LCD shows UNREGISTERED and the buzzer beeps rapidly. The GUI will show a registration dialog to add the new employee.

7. Managing Employees (users.json)

The users.json file stores all registered employees and their card UIDs. It is stored in the same folder as gui_reader.py. You can edit it directly with any text editor, or manage it through the GUI.

6.1 JSON Format

```
{
    "CARD UID HERE": {
        "name": "Full Name",
        "id": "Employee ID Number",
        "position": "Job Title"
    }
}
```

6.2 Pre-loaded Employees

Card UID	Name	Employee ID	Position
`78 51 60 E7`	Kraky	N/A	N/A
`19 10 76 A6`	Darren Taclajan	01240056849	Manager
`AE 86 F0 06`	Jeramy Barrameda	01240035624	Secretary
`16 EC 83 B9`	John Floyd Concepcion	01240017866	Assistant Manager
`99 44 57 C4`	John Errol Banaaga	012400142568	Janitor

6.3 Registering a New Card via GUI

- Tap an unregistered card on the RFID reader.
- A registration dialog will appear in the GUI.
- Enter the employee's Full Name, Employee ID, and Position.
- Click Save & Register. The entry is immediately saved to users.json.

6.4 Unregistering an Employee

- Click the Unregister Employee button in the main GUI window.
- Type the Card UID of the employee to remove.
- Click Confirm Unregister. The entry is permanently removed from users.json.

8. Troubleshooting

Problem	Solution
Arduino DISCONNECTED shown in red footer	Check that the Arduino is plugged in, and that the COM port in gui_reader.py matches Device Manager. Ensure Arduino Serial Monitor is CLOSED.
'Failed to connect to COM6' error	The Arduino Serial Monitor is still open, OR the COM port is wrong. Close the Serial Monitor and verify the port in Device Manager.
Database Status: Offline (JSON Mode Only)	XAMPP MySQL is not running. Open XAMPP Control Panel and click START next to MySQL.
LCD shows 'System Error / App Offline'	The Arduino sent the UID but the Python app did not respond in time (1.5 second timeout). Make sure gui_reader.py is running before scanning cards.
'No module named serial' error	pyserial is not installed. Run: python -m pip install pyserial
'No module named PIL' error	Pillow is not installed. Run: python -m pip install Pillow
'No module named mysql.connector' error	Run: python -m pip install mysql-connector-python
Card UID not matching users.json entries	The UID format must match exactly (e.g., '78 51 60 E7'). UIDs are in uppercase with spaces between each byte pair. Use the GUI registration dialog to add cards correctly.
GUI window does not open / crashes immediately	Make sure XAMPP MySQL is running first, OR the app will still open in JSON-only mode. Check the Command Prompt for error messages.
Buzzer not beeping on access granted	Verify the buzzer is connected to Arduino Pin 3 and GND. Confirm the arduino_code.ino was uploaded successfully. Check for a startup beep when the Arduino powers on.

9. Quick-Start Checklist (Every Time You Use the System)

Once setup is complete, follow this checklist every time you start the system:

1. Plug in the Arduino Uno via USB.
2. Open XAMPP Control Panel — START Apache and MySQL.
3. Open Command Prompt and navigate to the project folder.
4. Run: python gui_reader.py
5. Confirm the green 'Arduino Connected' bar is visible in the footer.
6. Confirm the header shows 'Database Status: Connected (Local)'.
7. Start scanning RFID cards!